

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №6
Дисциплина: Платформено-независимое программирование

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и информационные технологии

Преподаватель: _____ Шиян В.И.

1. Постановка задачи

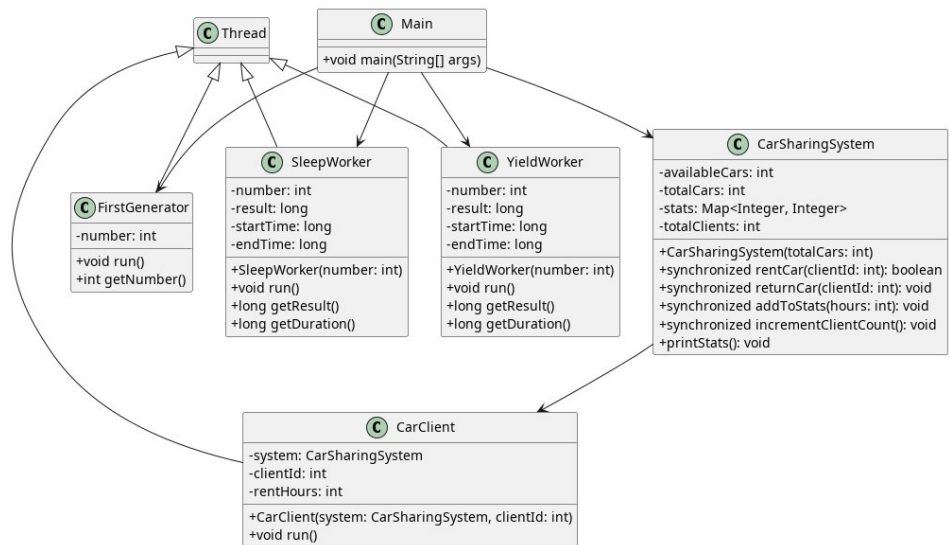
Часть 1. Каршеринг

- 30 автомобилей.
- Каждый час (модельного времени) приходит клиент — новый поток.
- Клиент берёт машину на случайное чётное время от 4 до 48 часов.
- Через указанное время возвращает.
- За сутки (24 часа) подсчитать количество клиентов.
- Вывести статистику по интервалам аренды (4, 6, 8, ..., 48 часов).
- Использовать `synchronized`, `wait()`, `notify()`.

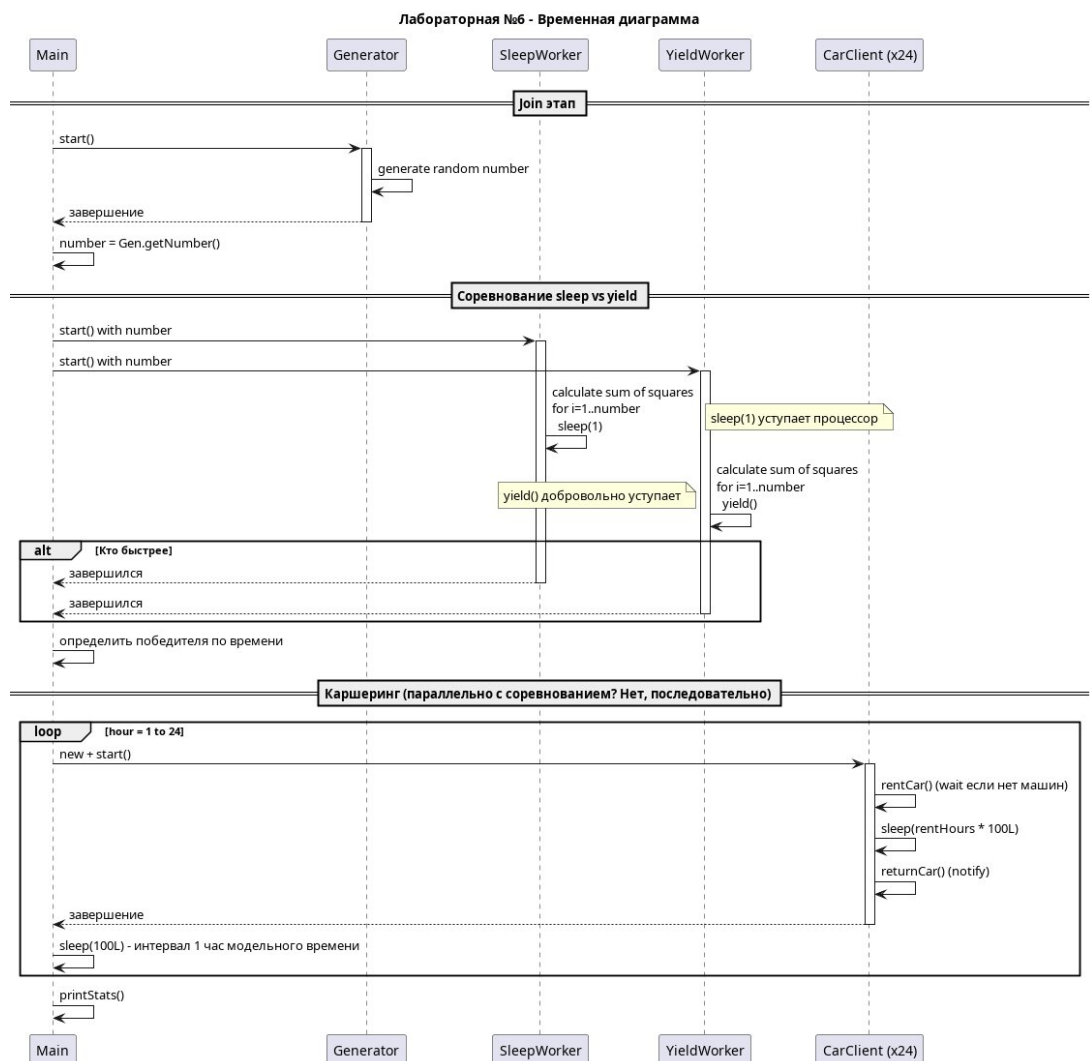
Часть 2. Join, yield, sleep

- Главный поток ждёт первый рабочий через `join()`.
- Первый рабочий генерирует случайное число (от 10 до 50).
- Главный передаёт это число второму рабочему.
- Первый и второй рабочие **соревнуются**, выполняя **одинаковое задание** (сумма квадратов от 1 до N).
- Первый рабочий использует `sleep(1)` на каждой итерации.
- Второй рабочий использует `yield()` на каждой итерации.
- Определить победителя (кто закончил первым).

2. UML-диаграмма классов



3. Временная диаграмма



4. Текст программы

CarSharingSystem.java

```
import java.util.Мap;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Система каршеринга с синхронизацией через synchronized/wait/notify
 */
public class CarSharingSystem {
    private int availableCars;
    private final int totalCars;
    private final Map<Integer, Integer> stats = new ConcurrentHashMap<>();
    private int totalClients = 0;

    public CarSharingSystem(int totalCars) {
        this.totalCars = totalCars;
        this.availableCars = totalCars;
        for (int h = 4; h <= 48; h += 2) {
            stats.put(h, 0);
        }
    }

    /**
     * Аренда машины. Если машин нет - ждём через wait()
     */
    public synchronized boolean rentCar(int clientId, int rentHours) throws InterruptedException {
        while (availableCars == 0) {
            System.out.printf("[Клиент %d] Нет машин, ожидание...\n", clientId);
            wait();
        }
        availableCars--;
        System.out.printf("[Клиент %d] Взял машину на %d ч. Свободно: %d/%d\n",
            clientId, rentHours, availableCars, totalCars);
        return true;
    }

    /**
     * Возврат машины с notify()
     */
    public synchronized void returnCar(int clientId) {
        availableCars++;
    }
}
```

```

        System.out.printf("[Клиент %d] Вернул машину. Свободно: %d/%d\n",
            clientId, availableCars, totalCars);
        notify();
    }

    public synchronized void addToStats(int hours) {
        stats.merge(hours, 1, Integer::sum);
    }

    public synchronized void incrementClientCount() {
        totalClients++;
    }

    public synchronized void printStats() {
        System.out.println("\n=== Статистика каршеринга за сутки ===");
        System.out.println("Всего клиентов: " + totalClients);
        System.out.println("Распределение по длительности аренды:");
        for (int h = 4; h <= 48; h += 2) {
            int count = stats.get(h);
            if (count > 0) {
                System.out.printf("  %d ч: %d клиент(ов)\n", h, count);
            }
        }
    }
}

```

CarClient.java

```

import java.util.Random;

/**
 * Клиент каршеринга - поток
 */
public class CarClient extends Thread {
    private final CarSharingSystem system;
    private final int clientId;
    private final int rentHours;

    public CarClient(CarSharingSystem system, int clientId) {
        this.system = system;
        this.clientId = clientId;
        Random rand = new Random();
        // чётное число от 4 до 48
        this.rentHours = 4 + 2 * rand.nextInt(23);
    }
}

```

```

    }

    @Override
    public void run() {
        try {
            system.rentCar(clientId, rentHours);
            system.addToStats(rentHours);
            system.incrementClientCount();

            // Симуляция: 1 час = 100 мс
            Thread.sleep(rentHours * 100L);

            system.returnCar(clientId);
        } catch (InterruptedException e) {
            System.out.printf("[Клиент %d] Прерван\n", clientId);
            Thread.currentThread().interrupt();
        }
    }
}

```

FirstGenerator.java

```

import java.util.Random;

/**
 * Первый рабочий - генерирует число для join
 */
public class FirstGenerator extends Thread {
    private int number = 0;

    @Override
    public void run() {
        Random rand = new Random();
        number = 10 + rand.nextInt(41); // 10..50
        System.out.println("[Generator] Сгенерировал число: " + number);
    }

    public int getNumber() {
        return number;
    }
}

```

SleepWorker.java

```
/**
 * Рабочий с использованием sleep(1)
 */
public class SleepWorker extends Thread {
    private final int number;
    private long result = 0;
    private long startTime = 0;
    private long endTime = 0;

    public SleepWorker(int number) {
        this.number = number;
    }

    @Override
    public void run() {
        startTime = System.currentTimeMillis();
        System.out.printf("[SleepWorker] Начал вычисление суммы квадратов от 1
до %d\n", number);

        for (int i = 1; i <= number; i++) {
            result += (long) i * i;
            try {
                Thread.sleep(1);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }

        endTime = System.currentTimeMillis();
        System.out.printf("[SleepWorker] Закончил. Результат = %d, время = %d
мс\n",
            result, (endTime - startTime));
    }

    public long getResult() {
        return result;
    }

    public long getDuration() {
        return endTime - startTime;
    }
}
```

YieldWorker.java

```
/**
 * Рабочий с использованием yield()
 */
public class YieldWorker extends Thread {
    private final int number;
    private long result = 0;
    private long startTime = 0;
    private long endTime = 0;

    public YieldWorker(int number) {
        this.number = number;
    }

    @Override
    public void run() {
        startTime = System.currentTimeMillis();
        System.out.printf("[YieldWorker] Начал вычисление суммы квадратов от 1
до %d\n", number);

        for (int i = 1; i <= number; i++) {
            result += (long) i * i;
            Thread.yield();
        }

        endTime = System.currentTimeMillis();
        System.out.printf("[YieldWorker] Закончил. Результат = %d, время = %d
мс\n",
            result, (endTime - startTime));
    }

    public long getResult() {
        return result;
    }

    public long getDuration() {
        return endTime - startTime;
    }
}
```


Main.java

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("--- Часть 2: join / yield / sleep ---\n");

        FirstGenerator generator = new FirstGenerator();
        generator.start();
        generator.join(); // главный ждёт

        int number = generator.getNumber();
        System.out.println("[Main] Получил число: " + number);

        SleepWorker sleepWorker = new SleepWorker(number);
        YieldWorker yieldWorker = new YieldWorker(number);

        sleepWorker.start();
        yieldWorker.start();

        sleepWorker.join();
        yieldWorker.join();

        System.out.println("\n--- Результат соревнования ---");
        long sleepTime = sleepWorker.getDuration();
        long yieldTime = yieldWorker.getDuration();

        if (sleepTime < yieldTime) {
            System.out.println("Победил SleepWorker (sleep)!");
        } else if (yieldTime < sleepTime) {
            System.out.println("Победил YieldWorker (yield)!");
        } else {
            System.out.println("Ничья!");
        }
        System.out.printf("SleepWorker: %d мс, YieldWorker: %d мс\n\n", sleep-
Time, yieldTime);
        System.out.println("--- Часть 1: Каршеринг ---\n");
        CarSharingSystem system = new CarSharingSystem(30);
        int simulationHours = 24;
        Thread[] clients = new Thread[simulationHours];

        long startTime = System.currentTimeMillis();

        for (int hour = 0; hour < simulationHours; hour++) {
            clients[hour] = new CarClient(system, hour + 1);
            clients[hour].start();
        }
    }
}
```

```

        System.out.printf("[Час %d] Пришёл клиент %d\n", hour + 1, hour + 1);

        // 1 час модельного времени = 100 мс реального
        Thread.sleep(100);
    }

    // Ждём всех клиентов
    for (Thread client : clients) {
        client.join();
    }

    system.printStats();
}
}

```

Результаты работы программы:

JOIN / YIELD / SLEEP:

```

[Generator] Сгенерировал число: 22
[Main] Получил число: 22
[SleepWorker] Начал вычисление суммы квадратов от 1 до 22
[YieldWorker] Начал вычисление суммы квадратов от 1 до 22
[YieldWorker] Закончил. Результат = 3795, время = 29 мс
[SleepWorker] Закончил. Результат = 3795, время = 54 мс

```

РЕЗУЛЬТАТ СОРЕВНОВАНИЯ:

```

Победил YieldWorker (yield)!
SleepWorker: 54 мс, YieldWorker: 29 мс

```

КАРШЕРИНГ:

```

[Час 1] Пришёл клиент 1
[Клиент 1] Взял машину на 38 ч. Свободно: 29/30
[Час 2] Пришёл клиент 2
[Клиент 2] Взял машину на 8 ч. Свободно: 28/30
[Час 3] Пришёл клиент 3
[Клиент 3] Взял машину на 36 ч. Свободно: 27/30
[Час 4] Пришёл клиент 4
[Клиент 4] Взял машину на 22 ч. Свободно: 26/30
[Час 5] Пришёл клиент 5
[Клиент 5] Взял машину на 20 ч. Свободно: 25/30
[Час 6] Пришёл клиент 6
[Клиент 6] Взял машину на 14 ч. Свободно: 24/30
[Час 7] Пришёл клиент 7
[Клиент 7] Взял машину на 4 ч. Свободно: 23/30
[Час 8] Пришёл клиент 8
[Клиент 8] Взял машину на 30 ч. Свободно: 22/30

```

[Час 9] Пришёл клиент 9
[Клиент 9] Взял машину на 6 ч. Свободно: 21/30
[Клиент 2] Вернул машину. Свободно: 22/30
[Час 10] Пришёл клиент 10
[Клиент 10] Взял машину на 8 ч. Свободно: 21/30
[Клиент 7] Вернул машину. Свободно: 22/30
[Час 11] Пришёл клиент 11
[Клиент 11] Взял машину на 26 ч. Свободно: 21/30
[Час 12] Пришёл клиент 12
[Клиент 12] Взял машину на 42 ч. Свободно: 20/30
[Час 13] Пришёл клиент 13
[Клиент 13] Взял машину на 10 ч. Свободно: 19/30
[Час 14] Пришёл клиент 14
[Клиент 14] Взял машину на 10 ч. Свободно: 18/30
[Клиент 9] Вернул машину. Свободно: 19/30
[Час 15] Пришёл клиент 15
[Клиент 15] Взял машину на 36 ч. Свободно: 18/30
[Час 16] Пришёл клиент 16
[Клиент 16] Взял машину на 42 ч. Свободно: 17/30
[Час 17] Пришёл клиент 17
[Клиент 17] Взял машину на 10 ч. Свободно: 16/30
[Клиент 10] Вернул машину. Свободно: 17/30
[Час 18] Пришёл клиент 18
[Клиент 18] Взял машину на 28 ч. Свободно: 16/30
[Час 19] Пришёл клиент 19
[Клиент 19] Взял машину на 4 ч. Свободно: 15/30
[Клиент 6] Вернул машину. Свободно: 16/30
[Час 20] Пришёл клиент 20
[Клиент 20] Взял машину на 32 ч. Свободно: 15/30
[Час 21] Пришёл клиент 21
[Клиент 21] Взял машину на 10 ч. Свободно: 14/30
[Час 22] Пришёл клиент 22
[Клиент 22] Взял машину на 36 ч. Свободно: 13/30
[Клиент 13] Вернул машину. Свободно: 14/30
[Клиент 19] Вернул машину. Свободно: 15/30
[Час 23] Пришёл клиент 23
[Клиент 23] Взял машину на 42 ч. Свободно: 14/30
[Клиент 14] Вернул машину. Свободно: 15/30
[Час 24] Пришёл клиент 24
[Клиент 24] Взял машину на 4 ч. Свободно: 14/30
[Клиент 5] Вернул машину. Свободно: 15/30
[Клиент 4] Вернул машину. Свободно: 16/30
[Клиент 17] Вернул машину. Свободно: 17/30
[Клиент 24] Вернул машину. Свободно: 18/30
[Клиент 21] Вернул машину. Свободно: 19/30
[Клиент 11] Вернул машину. Свободно: 20/30
[Клиент 8] Вернул машину. Свободно: 21/30
[Клиент 1] Вернул машину. Свободно: 22/30
[Клиент 3] Вернул машину. Свободно: 23/30
[Клиент 18] Вернул машину. Свободно: 24/30

[Клиент 15] Вернул машину. Свободно: 25/30
[Клиент 20] Вернул машину. Свободно: 26/30
[Клиент 12] Вернул машину. Свободно: 27/30
[Клиент 16] Вернул машину. Свободно: 28/30
[Клиент 22] Вернул машину. Свободно: 29/30
[Клиент 23] Вернул машину. Свободно: 30/30

СТАТИСТИКА КАРШЕРИНГА ЗА СУТКИ:

Всего клиентов: 24

Распределение по длительности аренды:

4 ч: 3 клиент(ов)
6 ч: 1 клиент(ов)
8 ч: 2 клиент(ов)
10 ч: 4 клиент(ов)
14 ч: 1 клиент(ов)
20 ч: 1 клиент(ов)
22 ч: 1 клиент(ов)
26 ч: 1 клиент(ов)
28 ч: 1 клиент(ов)
30 ч: 1 клиент(ов)
32 ч: 1 клиент(ов)
36 ч: 3 клиент(ов)
38 ч: 1 клиент(ов)
42 ч: 3 клиент(ов)